Unsupervised learning: k-means

CSCI-P 556

ZORAN TIGANJ

Announcements

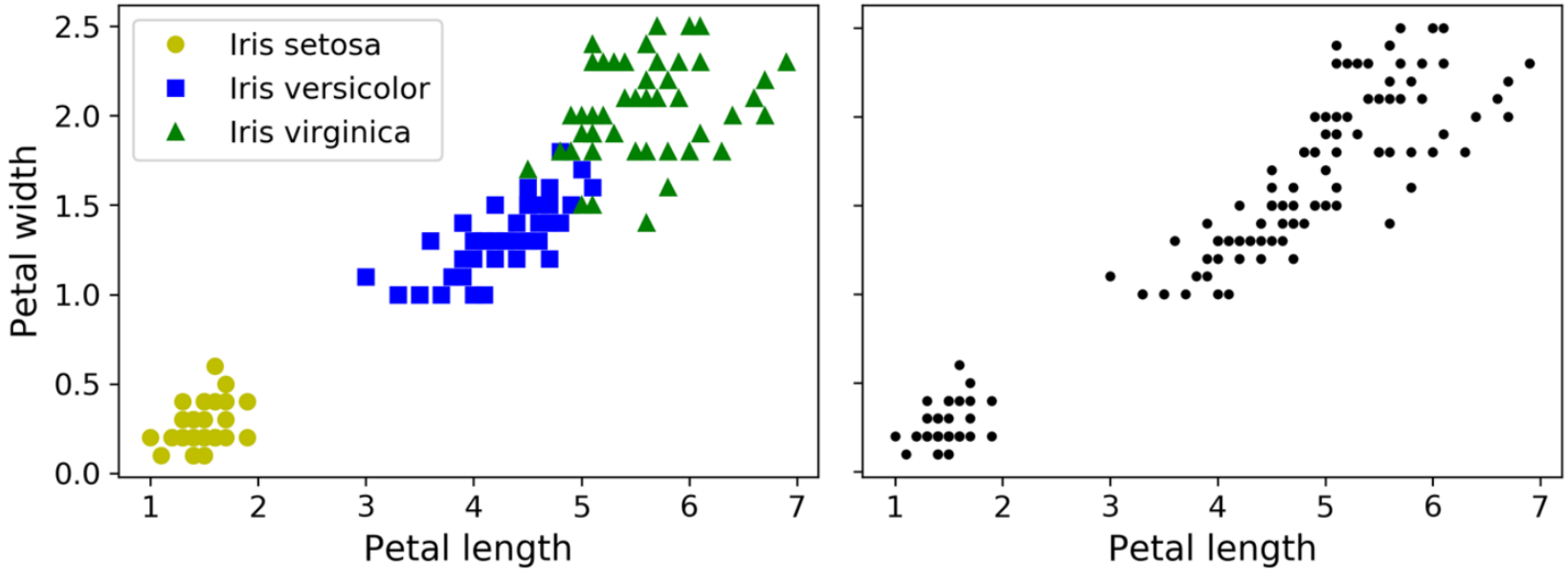
- HW2 due on Friday (HW3 will be released shortly after)
- HW1Q1 grades are posted, HW1Q2 grades will be posted in a few days.

Today and next class: unsupervised learning

- Although most of the applications of Machine Learning today are based on supervised learning, the vast majority of the available data is unlabeled: we have the input features **X**, but we do not have the labels **y**.
- Example: anomaly detection on a production line.
- Unsupervised learning often involves *clustering*
  - The goal is to group similar instances together into clusters.

Clustering

Classification (left) vs clustering (right)

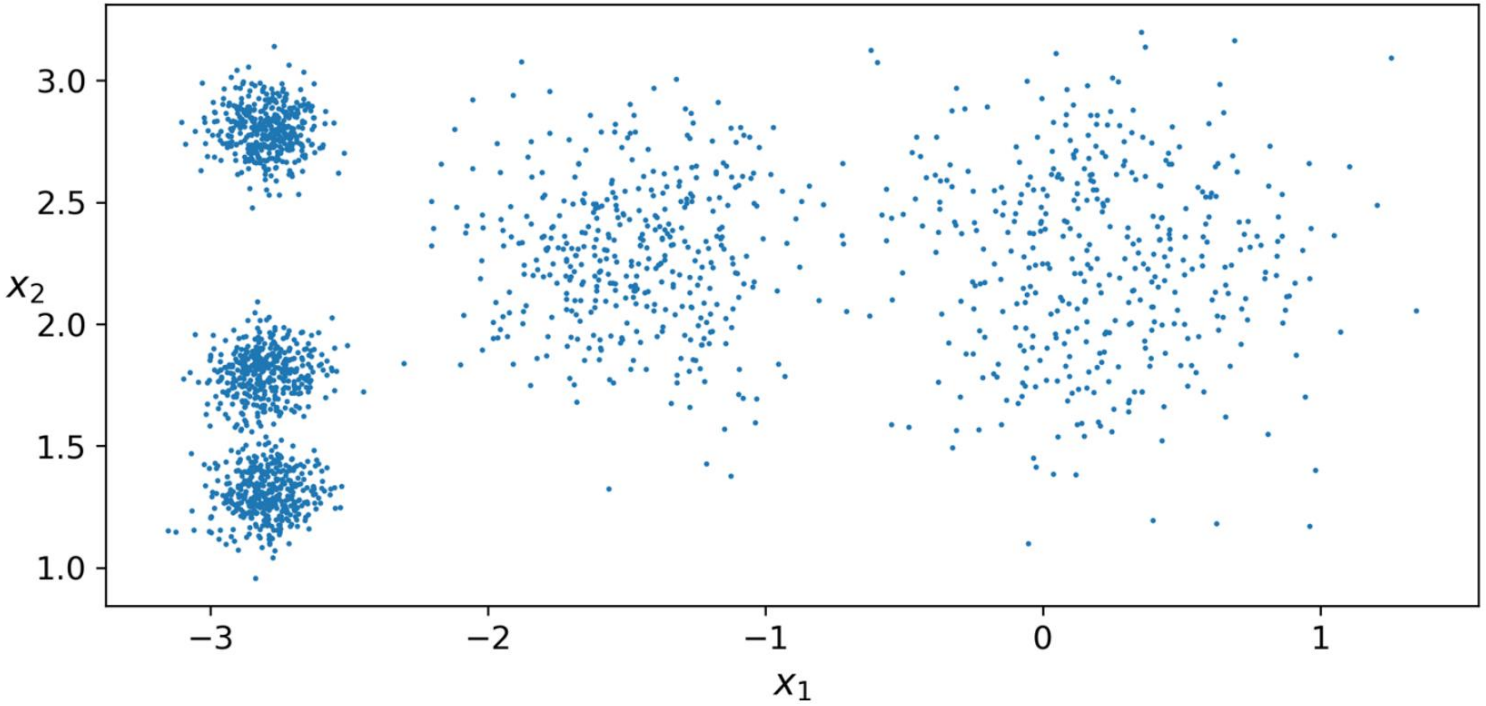


Applications of clustering

- For customer segmentation
  - E.g., customer segmentation in recommender systems to suggest content that other users in the same cluster enjoyed.
- For data analysis
  - When you analyze a new dataset, it can be helpful to run a clustering algorithm, and then analyze each cluster separately
- As a dimensionality reduction technique
  - Once a dataset has been clustered, it is usually possible to measure each instance's affinity with each cluster (affinity is any measure of how well an instance fits into a cluster). Each instance's feature vector x can then be replaced with the vector of its cluster affinities.
- For anomaly detection
  - Any instance that has a low affinity to all the clusters is likely to be an anomaly.

K-Means

- How many clusters do you see?

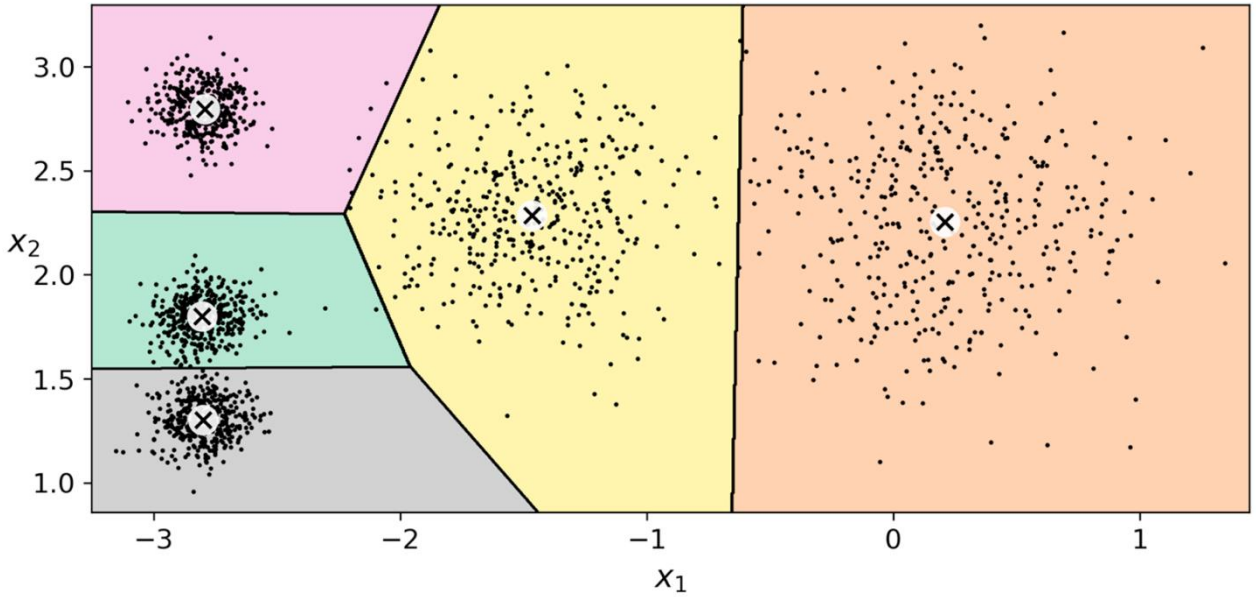


K-Means

- Let's train a K-Means clustering on this dataset. It will try to find each blob's center and assign each instance to the closest blob:

```
from sklearn.cluster import KMeans
k = 5
kmeans = KMeans(n_clusters=k)
y_pred = kmeans.fit_predict(X)
```

```
>>> kmeans.cluster_centers_
array([[-2.80389616,  1.80117999],
       [ 0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [-2.80037642,  1.30082566]])
```




```
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])
>>> kmeans.predict(X_new)
array([1, 1, 2, 2], dtype=int32)
```

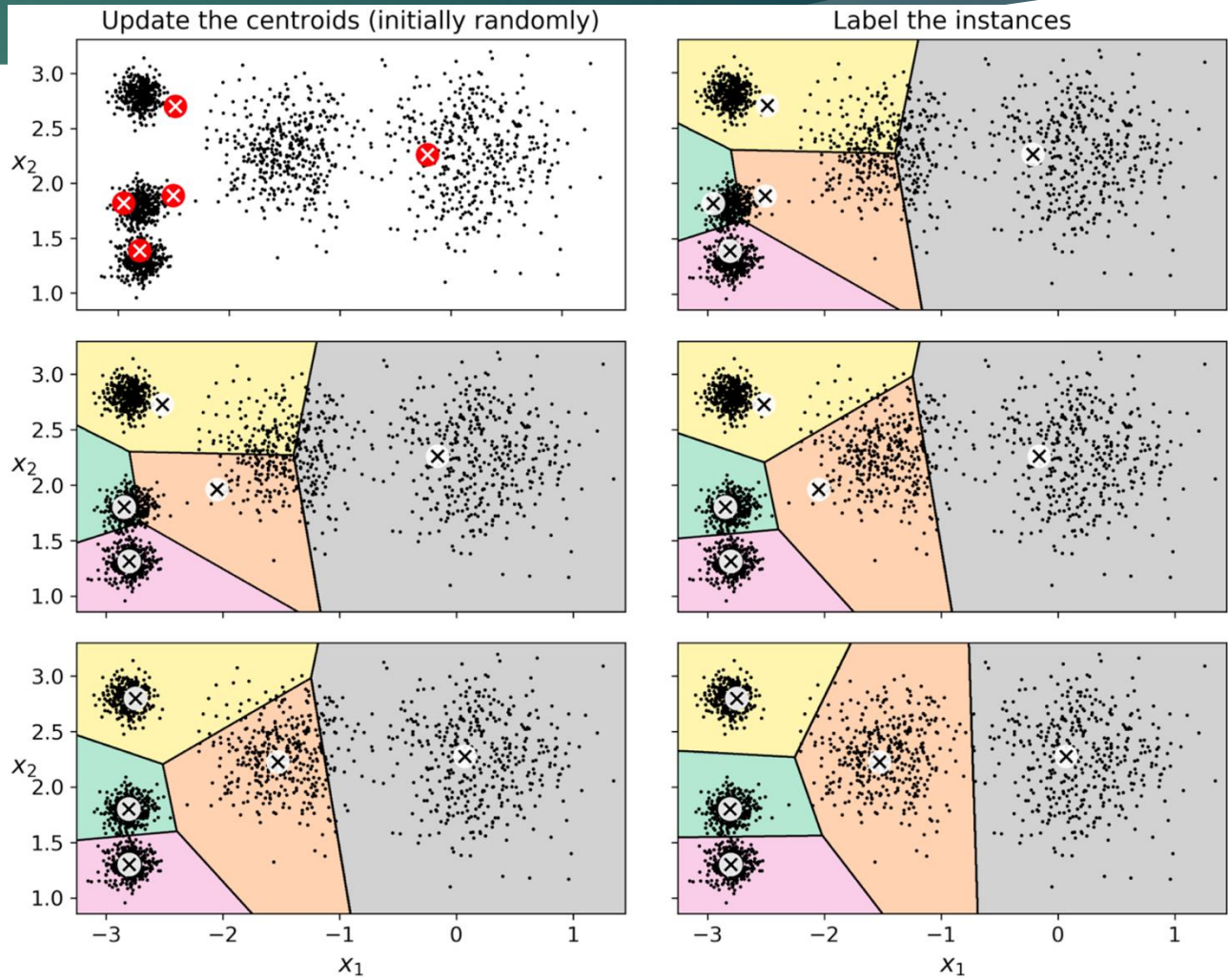
K-Means

- Instead of assigning each instance to a single cluster, which is called **hard clustering**, it can be useful to give each instance a score per cluster, which is called **soft clustering**.
- In the KMeans class, the transform() method measures the distance from each instance to every centroid:

```
>>> kmeans.transform(X_new)
array([[2.81093633, 0.32995317, 2.9042344 , 1.49439034, 2.88633901],
       [5.80730058, 2.80290755, 5.84739223, 4.4759332 , 5.84236351],
       [1.21475352, 3.29399768, 0.29040966, 1.69136631, 1.71086031],
       [0.72581411, 3.21806371, 0.36159148, 1.54808703, 1.21567622]])
```

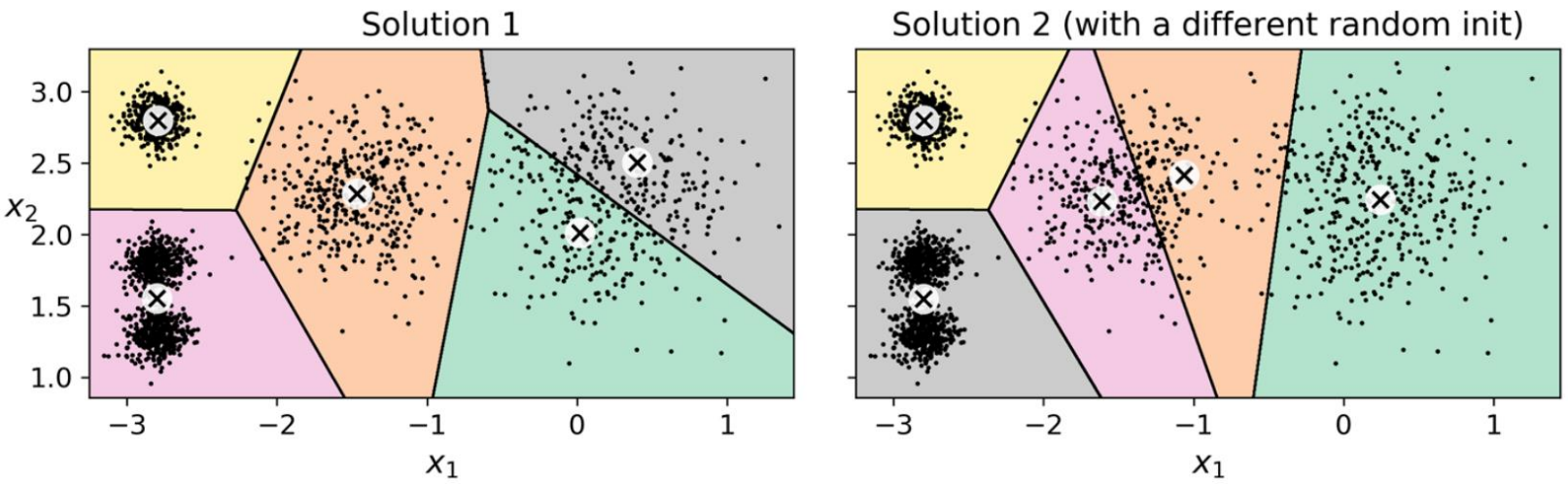
The K-Means algorithm

- The K-Means algorithm
  - Start by placing the centroids randomly (e.g., by picking k instances at random and using their locations as centroids).
  - Then label the instances
  - update the centroids, $\boldsymbol{\mu}_i = \frac{1}{|S_i|} \sum_{\mathbf{x} \in S_i} \mathbf{x}$
  - label the instances,
  - update the centroids,
  - and so on until the centroids stop moving.
  - The algorithm is guaranteed to converge in a finite number of steps.



The K-Means algorithm

- Although the algorithm is guaranteed to converge, it may not converge to the right solution (i.e., it may converge to a local optimum): whether it does or not depends on the centroid initialization.
- Suboptimal solutions due to unlucky centroid initializations:



Centroid initialization methods

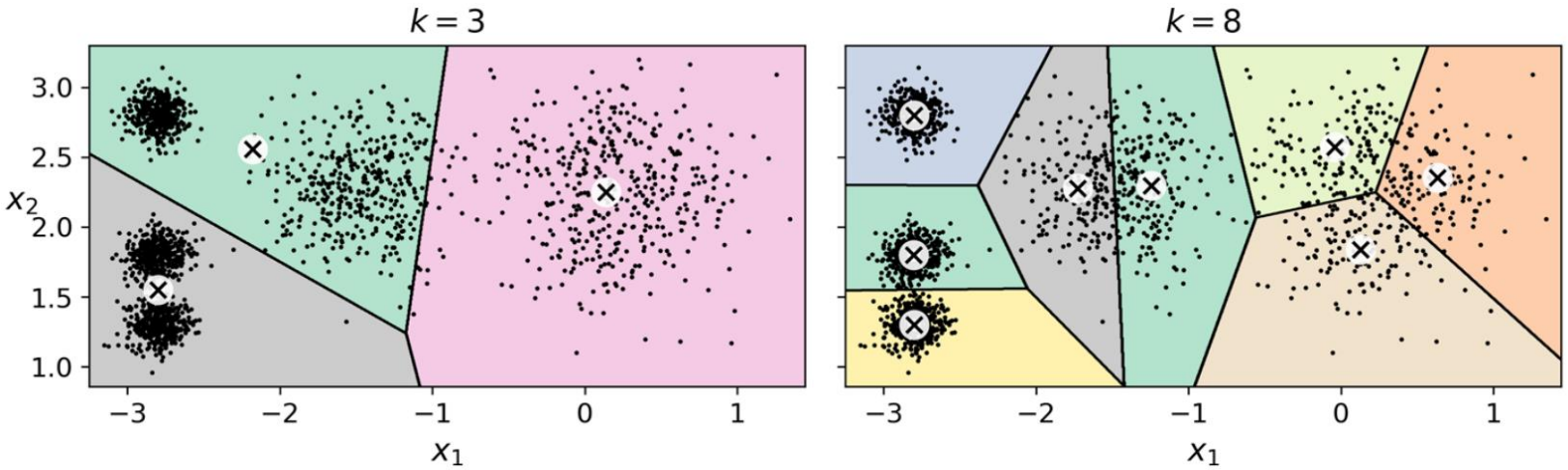
- Let's look at a few ways you can mitigate this risk by improving the centroid initialization:
  - If you happen to know approximately where the centroids should be, you can start from there.
  - Run the algorithm multiple times with different random initializations and keep the best solution (in terms of mean squared distance between each instance and its closest centroid, which is also called model **intertia**).
  - K-Means++: selects centroids that are distant from one another:
    - Take one centroid $\mathbf{c}^{(1)}$, chosen uniformly at random from the dataset.
    - Take a new centroid $\mathbf{c}^{(i)}$, choosing an instance $\mathbf{x}^{(i)}$ with probability

$$D\left(\mathbf{x}^{(i)}\right)^2 / \sum_{j=1}^{m} D\left(\mathbf{x}^{(j)}\right)^2$$

      - where $D(\mathbf{x}^{(i)})$ is the distance between the instance $\mathbf{x}^{(i)}$ and the closest centroid that was already chosen.
      - This probability distribution ensures that instances farther away from already chosen centroids are much more likely be selected as centroids.

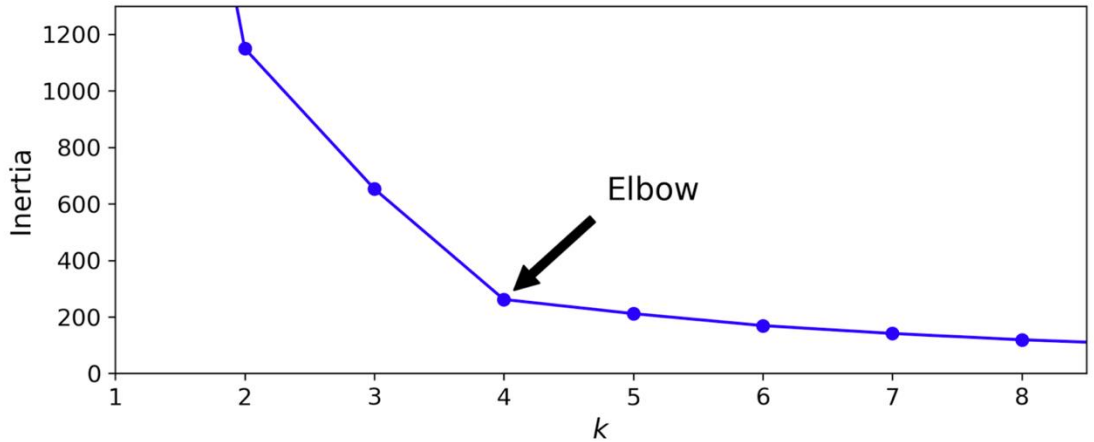
Finding the optimal number of clusters

- So far, we have set the number of clusters k to 5 because it was obvious by looking at the data that this was the correct number of clusters.
- But in general, it will not be so easy to know how to set k, and the result might be quite bad if you set it to the wrong value.



Finding the optimal number of clusters

- Can we use model inertial (mean squared distance between each instance and its closest centroid)?
- The inertia is not a good performance metric when trying to choose k because it keeps getting lower as we increase k.
- The below curve has roughly the shape of an arm, and there is an "elbow" at k = 4.



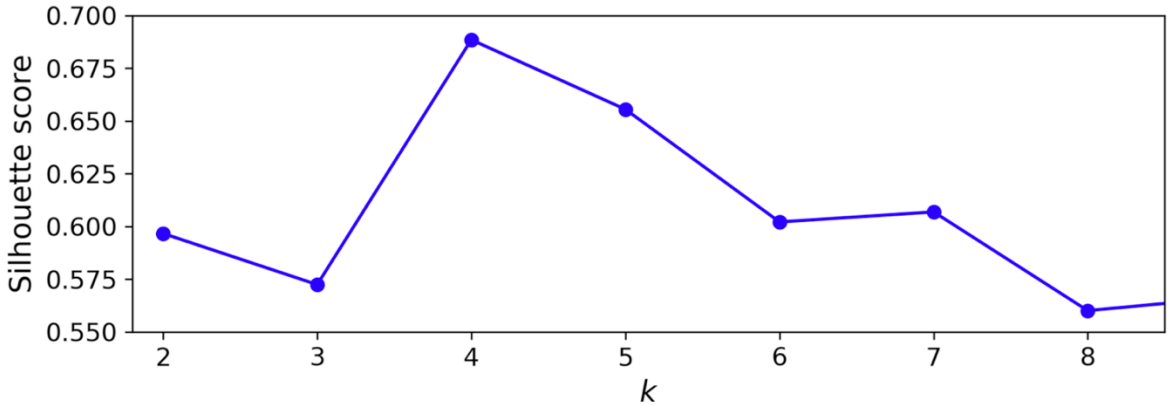
Finding the optimal number of clusters

- A more precise approach than using elbow is to use the silhouette score, which is the mean silhouette coefficient over all the instances.
- An instance's silhouette coefficient is equal to (b – a) / max(a, b),
  - where a is the mean distance to the other instances in the same cluster (i.e., the mean intra-cluster distance) and
  - b is the mean nearest-cluster distance (i.e., the mean distance to the instances of the next closest cluster).
- The silhouette coefficient can vary between –1 and +1.
  - A coefficient close to +1 means that the instance is well inside its own cluster and far from other clusters,
  - a coefficient close to 0 means that it is close to a cluster boundary,
  - a coefficient close to –1 means that the instance may have been assigned to the wrong cluster.

Finding the optimal number of clusters

- To compute the silhouette score, you can use Scikit-Learn's silhouette_score() function, giving it all the instances in the dataset and the labels they were assigned

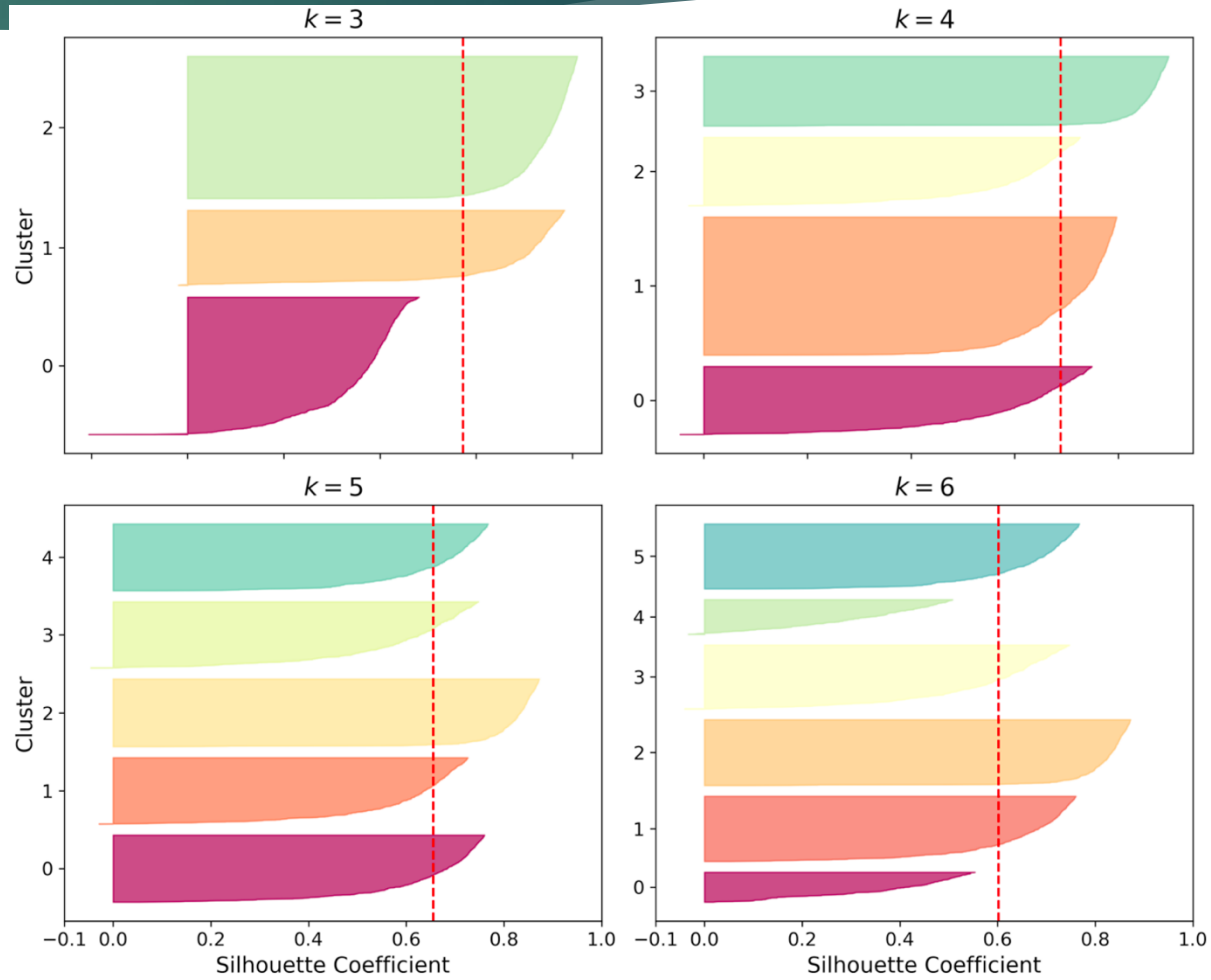
```
>>> from sklearn.metrics import silhouette_score
>>> silhouette_score(X, kmeans.labels_)
0.655517642572828
```



This visualization is much richer than the previous one: although it confirms that k = 4 is a very good choice, it also underlines the fact that k = 5 is quite good as well, and much better than k = 6 or 7.

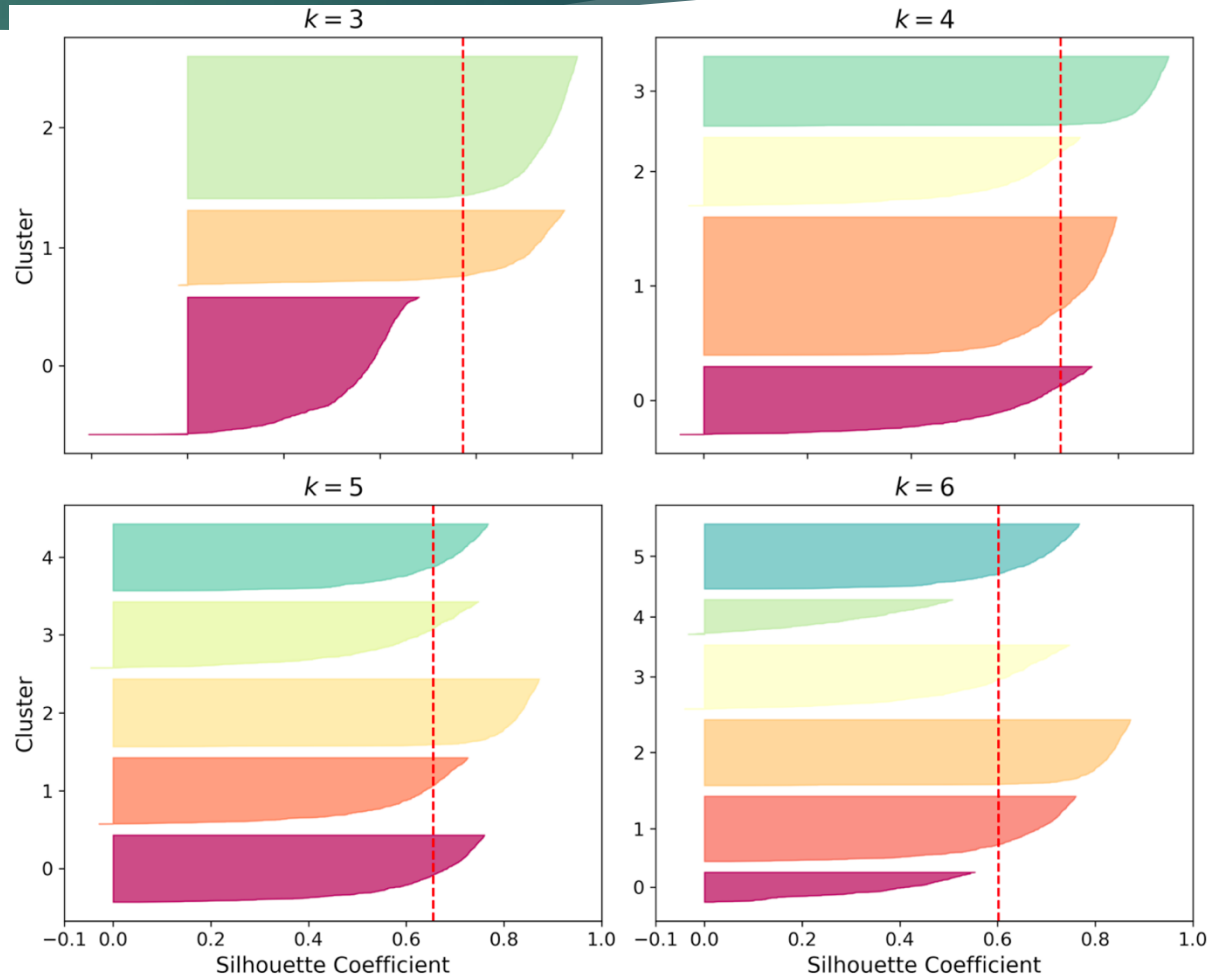
Finding the optimal number of clusters

- An even more informative visualization is obtained when you plot every instance's silhouette coefficient, sorted by the cluster they are assigned to and by the value of the coefficient. This is called a **silhouette diagram**.
- The vertical dashed lines represent the silhouette score for each number of clusters.
- When most of the instances in a cluster have a lower coefficient than this score then the cluster is rather bad since this means its instances are much too close to other clusters (e.g., 3 and 6 have bad clusters).



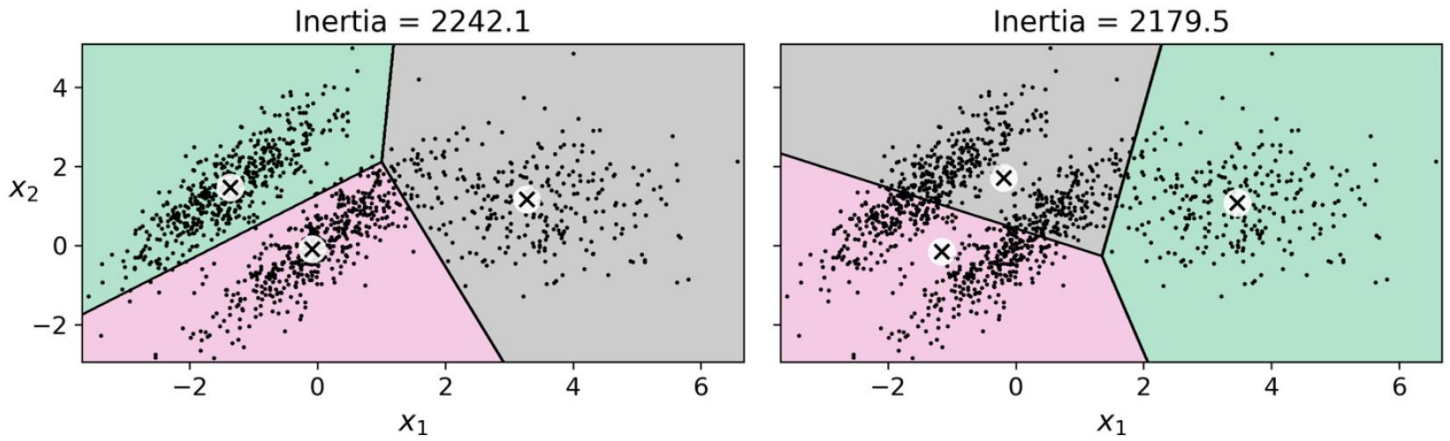
Finding the optimal number of clusters

- When k = 4, the cluster at index 1 (the third from the top) is rather big.
- When k = 5, all clusters have similar sizes.
- So, even though the overall silhouette score from k = 4 is slightly greater than for k = 5, it seems like a good idea to use k = 5 to get clusters of similar sizes.



Limits of K-Means

- Despite its many merits, most notably being fast and scalable, K-Means is not perfect.
- It is necessary to run the algorithm several times to avoid suboptimal solutions, plus you need to specify the number of clusters, which can be quite a hassle.
- Moreover, K-Means does not behave very well when the clusters have varying sizes, different densities, or nonspherical shapes.



It is important to scale the input features before you run K-Means, or the clusters may be very stretched and K-Means will perform poorly.

Using Clustering for Image Segmentation

- Image segmentation is the task of partitioning an image into multiple segments.
- We will do a simple version of image segmentation: color segmentation: assign pixels to the same segment if they have a similar color.
  - This can be useful if you want to analyze satellite images to measure how much total forest area there is in a region.
  - For more complicated image segmentation (e.g., identifying a pedestrian from a self-driving car, we would use convolutional neural networks).

Original image

Using Clustering for Image Segmentation

- First, use Matplotlib's imread() function to load the image

```
>>> from matplotlib.image import imread  # or `from imageio import imread`
>>> image = imread(os.path.join("images","unsupervised_learning","ladybug.png"))
```

```
>>> image.shape
(533, 800, 3)
```

The first dimension's size is the height; the second is the width; and the third is the number of color channels, in this case red, green, and blue (RGB).

Original image

Using Clustering for Image Segmentation

- Result: It may identify a color cluster for all shades of green. Next, for each color (e.g., dark green), it looks for the mean color of the pixel's color cluster. For example, all shades of green may be replaced with the same light green color (assuming the mean color of the green cluster is light green).

Using Clustering for Image Segmentation

- The following code reshapes the array to get a long list of RGB colors, then it clusters these colors using K-Means:

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8).fit(X)
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
segmented_img = segmented_img.reshape(image.shape)
```

- Result: when you use fewer than eight clusters, notice that the ladybug' s flashy red color fails to get a cluster of its own: it gets merged with colors from the environment. This is because K-Means prefers clusters of similar sizes. The ladybug is small—much smaller than the rest of the image—so even though its color is flashy, K-Means fails to dedicate a cluster to it.

Using Clustering for Preprocessing

- Clustering can be an efficient approach to dimensionality reduction, in particular as a preprocessing step before a supervised learning algorithm.
- As an example of using clustering for dimensionality reduction, let's tackle the digits dataset, which is a simple MNIST-like dataset containing 1,797 grayscale 8 × 8 images representing the digits 0 to 9.
- First, load the dataset:

```
from sklearn.datasets import load_digits

X_digits, y_digits = load_digits(return_X_y=True)
```

Using Clustering for Preprocessing

- Clustering can be an efficient approach to dimensionality reduction, in particular as a preprocessing step before a supervised learning algorithm.
- As an example of using clustering for dimensionality reduction, let's tackle the digits dataset, which is a simple MNIST-like dataset containing 1,797 grayscale 8 × 8 images representing the digits 0 to 9.
- Now, split it into a training set and a test set:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_digits, y_digits)
```

Using Clustering for Preprocessing

- Clustering can be an efficient approach to dimensionality reduction, in particular as a preprocessing step before a supervised learning algorithm.
- As an example of using clustering for dimensionality reduction, let's tackle the digits dataset, which is a simple MNIST-like dataset containing 1,797 grayscale 8 × 8 images representing the digits 0 to 9.
- Next, fit a Logistic Regression model:

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)
```

Using Clustering for Preprocessing

- Clustering can be an efficient approach to dimensionality reduction, in particular as a preprocessing step before a supervised learning algorithm.
- As an example of using clustering for dimensionality reduction, let's tackle the digits dataset, which is a simple MNIST-like dataset containing 1,797 grayscale 8 × 8 images representing the digits 0 to 9.
- Let's evaluate its accuracy on t he test set:

```
>>> log_reg.score(X_test, y_test)
0.9688888888888889
```

Using Clustering for Preprocessing

- Let's see if we can do better by using K-Means as a preprocessing step.
- We will create a pipeline that will first cluster the training set into 50 clusters and replace the images with their distances to these 50 clusters, then apply a Logistic Regression model:

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("kmeans", KMeans(n_clusters=50)),
    ("log_reg", LogisticRegression()),
])
pipeline.fit(X_train, y_train)

>>> pipeline.score(X_test, y_test)
0.9777777777777777
```

Using Clustering for Preprocessing

- We chose the number of clusters k arbitrarily; we can surely do better.
- Since K-Means is just a preprocessing step in a classification pipeline, finding a good value for k is much simpler than earlier. There's no need to perform silhouette analysis or minimize the inertia;
- The best value of k is simply the one that results in the best classification performance during cross-validation. We can use GridSearchCV to find the optimal number of clusters:

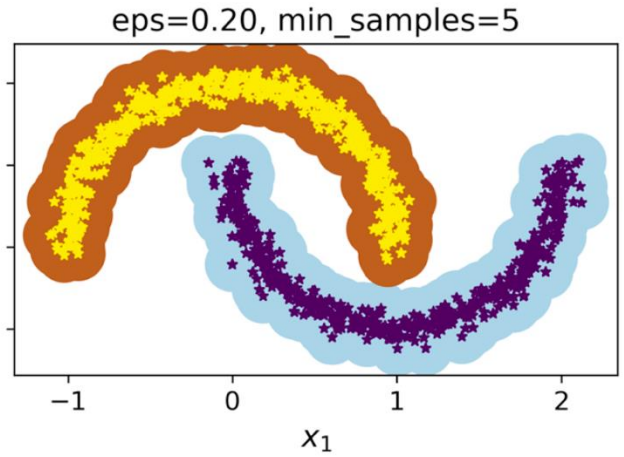
```
from sklearn.model_selection import GridSearchCV

param_grid = dict(kmeans__n_clusters=range(2, 100))
grid_clf = GridSearchCV(pipeline, param_grid, cv=3, verbose=2)
grid_clf.fit(X_train, y_train)
```

```
>>> grid_clf.best_params_
{'kmeans__n_clusters': 99}
>>> grid_clf.score(X_test, y_test)
0.9822222222222222
```

Other unsupervised clustering methods

- DBSCAN
  - This algorithm defines clusters as continuous regions of high density. It works well if all the clusters are dense enough and if they are well separated by density regions.
- Agglomerative clustering
  - A hierarchy of clusters is built from the bottom up.
- BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies)
  - Algorithm was designed specifically for very large datasets, and it can be faster than K-Means, with similar results, as long as the number of features is not too large (<20).
- Mean-Shift
  - This algorithm starts by placing a circle centered on each instance; then for each circle it computes the mean of all the instances located within it, and it shifts the circle so that it is centered on t he mean. Next, it iterates this mean-shifting step until all the circles stop moving.



Other unsupervised clustering methods

- Affinity propagation
  - This algorithm uses a voting system, where instances vote for similar instances to be their representatives, and once the algorithm converges, each representative and its voters form a cluster.
- Spectral clustering
  - This algorithm takes a similarity matrix between the instances and creates a low-dimensional embedding from it (i.e., it reduces its dimensionality), then it uses another clustering algorithm in this lowdimensional space (Scikit-Learn's implementation uses K-Means).
- Gaussian mixture models (next lecture)
  - Can be used for density estimation, clustering, and anomaly detection.

Next time

- Gaussian mixture models, from Chapter 9, *Hands on machine learning textbook*